

UNIVERSITÀ DEGLI STUDI
DI GENOVA

DIPARTIMENTO DI INGEGNERIA NAVALE, ELETTRICA, ELETTRONICA E DELLE TELECOMUNICAZIONI

Corso di Laurea in Ingegneria Elettronica e Tecnologie dell'Informazione

Sistema embedded per controllo di telescopio in osservatorio astronomico

Tesi di laurea triennale

14 OTTOBRE 2026

Relatore:
Prof. Rodolfo Zunino
Correlatore:
Prof. Andrea Marenco

Candidato: *Mauro Gori*

Sommario

L’obbiettivo di questa tesi è quello di documentare il processo di progettazione, sviluppo e implementazione del sistema di controllo e remotizzazione dell’osservatorio LISA del Liceo Scientifico Statale G.D. Cassini.

In particolare, questo lavoro si è concentrato sul modulo per interfacciare il firmware di controllo del telescopio Meade LX200GPS ACF AutoStar II con il Raspberry Pi 3b programmato per svolgere le funzioni di gestione di tutti i moduli all’interno dell’osservatorio tramite WiFi.

Inizialmente sono state eseguite delle prove di funzionamento del telescopio tramite un convertitore commerciale da RS232 a USB, che sono risultate positive, successivamente è stato collegato un microcontrollore ESP32 con un modulo di conversione montante chip MAX3232, anche esso normalmente in commercio. Dato il risultato positivo del secondo passaggio, è iniziato lo sviluppo della scheda elettronica e del firmware caricato sul microcontrollore.

Infine il sistema è stato assemblato in un apposito contenitore di plastica appositamente costruito di dimensioni 15x12x8cm, dotato di interruttore di accensione e tasto di reset del microcontrollore.

Indice

Sommario **2**
1 Introduzione **4**
1.1 Situazione Iniziale 4
1.2 Obiettivi 5
Specifiche Hardware 5
Specifiche Software 5
2 Strumenti utilizzati **6**
2.1 ESP32 DEV KIT V1 6
2.2 Convertitore RS232 a TTL con MAX3232 6
2.3 Scheda PCB 6
2.4 Cavo USB-RS232 7
2.5 HTTP 7
2.6 API REST 8
2.7 TCP 8
2.8 PuTTY 8
2.9 KiCAD 9
2.10 Visual Studio Code 9
3 Architettura del sistema **10**
3.1 Topologia del sistema 10
3.2 Modalità di interconnessione 11
4 Il sistema telescopio **12**
4.1 La porta seriale RS232 12
4.2 Il protocollo Meade LX200 14
4.3 Nota sulla codifica dei caratteri 16
5 Il firmware del microcontrollore **18**
5.1 Controllo del telescopio 18
Struct AltAz 18
Funzioni LX200 19
Le stringhe di comando 19
5.2 Gestione della rete 20
Interfaccia REST API 20
Trasmissione dei dati asincrona via TCP e JSON 21
6 Contributo personale e considerazioni conclusive **22**
Riferimenti Bibliografici **23**

1 Introduzione

1.1 Situazione Iniziale

Il Liceo Scientifico Gian Domenico Cassini è dotato di un osservatorio astronomico, posto sul lastrico solare dell'edificio costruito nei primi anni 2000. Esso è dotato di una cupola rotante azionata da un motore a corrente continua, montato sulla cupola c'è un portellone il quale si chiude quando l'osservatorio è inattivo. Al centro della struttura si trova il telescopio Meade LX200GPS ACF AutoStarII. Originariamente l'osservatorio era automatizzato grazie ad una scheda PICPLC 16A la quale sincronizzava la posizione del telescopio con la cupola rotante e gestiva autonomamente l'apertura e chiusura del portellone. Tuttavia, nel momento in cui ho avuto accesso al telescopio, la scheda era guasta e il sistema di sincronizzazione non era funzionante ormai da anni.



Figura 1: Osservatorio astronomico LISA

Nel 2025 era stato realizzato un sistema di controllo del motore che permette la rotazione della cupola, successivamente è stato inoltre sviluppato un software di coordinamento e accesso remoto a tutti gli elementi che compongono l'osservatorio. Si è reso necessario quindi rendere disponibile un canale di comunicazione con il telescopio, oggetto di questa tesi.

1.2 Obiettivi

L'obbiettivo fondamentale del mio lavoro è stato quello di progettare e realizzare una scheda stampata per interfacciare e collegare il telescopio tramite WiFi al sistema di controllo centrale.

Specifiche Hardware

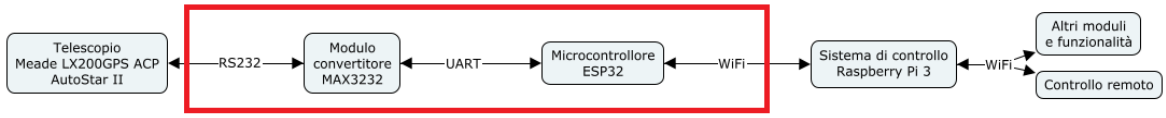


Figura 2: Schematico Hardware sviluppato

Il microcontrollore deve essere montato in maniera sicura sulla struttura del telescopio, in modo tale da connettere il suo cavo seriale direttamente ad un modulo convertitore. Non essendo isolata termicamente dall'esterno la struttura è soggetta a sbalzi di temperatura e infiltrazioni di umidità, inoltre deve essere dotata di un'interfaccia fisica molto semplice e robusta, adatta alle condizioni operative di un luogo pubblico come una scuola.
Per soddisfare tali necessità è stato deciso di assemblare il sistema all'interno di una scatola di plastica per impianti elettrici, dotata di guarnizione isolante e resistente. L'impianto altresì fornisce delle funzionalità di base semplificate che sono garantite da un interruttore per l'alimentazione e un pulsante per effettuare il reset. Il sistema deve essere alimentato da un'alimentazione esterna a 5v CC.

Specifiche Software

Il firmware del microcontrollore ha poche ma fondamentali funzionalità che ne assicurano il corretto funzionamento. Nello specifico:

- **Connettersi ad una rete WiFi locale**
- **Fornire informazioni come risorse HTTP**: il sistema fornisce alla rete locale del sistema di controllo centrale una struttura http, dalla quale è possibile richiedere informazioni e impartire ordini da far eseguire al telescopio.
- **Scambiare informazioni con il telescopio**: utilizzare il modulo traslatore di livello per inviare stringhe di comando sulla porta seriale, e leggere le stringhe di risposta provenienti dal telescopio.
- **Effettuare il parsing delle informazioni e rilevare eventuali errori**: le stringhe ricevute dal telescopio non possono essere inviate al sistema centrale così come sono, si rende necessario quindi effettuare un parsing in oggetti JSON, riconoscendo e segnalando eventuali errori, che siano richieste non eseguibili o problemi con il telescopio.
- **Inviare i payload di informazioni come pacchetti TCP**: i file JSON sono quindi inviati al sistema centrale per essere processati.

2 Strumenti utilizzati

2.1 ESP32 DEV KIT V1

Il microcontrollore utilizzato in questo progetto viene commercializzato sui principali siti di e-commerce e distributori di componenti elettronici in un pacchetto a 30 pin, con integrato il chip di comunicazione USB per programmarli e un regolatore di tensione da 5v dell'alimentazione USB a 3.3v tensione operativa del microcontrollore.
È installata un'antenna per le comunicazioni WiFi e Bluetooth direttamente sulla scheda, semplificando molto l'integrazione di connettività wireless ed eliminando il ricorso a moduli esterni, tipo i comuni NRF24.



Figura 3: ESP32 DevKit-V1

La scelta di questo microcontrollore in questo formato è stata dettata dall'ampia disponibilità e dal basso costo forniti dal mercato; l'ampia diffusione di questi microcontrollori garantisce il facile reperimento di risorse software e supporto tecnico in caso si riscontrassero problemi durante lo sviluppo. Inoltre, nonostante sia il modello base di questa linea di microntrollori, esso viaggia ad una velocità di clock da 240MHz su due core e 4MB di memoria RAM, fornendo inoltre accesso a bus I2C, SPI, e a due periferiche UART.

2.2 Convertitore RS232 a TTL con MAX3232

Per convertire le tensioni di un canale RS232 a TTL (Transistor to Transistor logic, approfondito in seguito) è disponibile in commercio un modulo traslatore di livello che sfrutta il circuito integrato MAX3232, dispone dell'attacco DB-9 standard per questo tipo di connessioni e un attacco PinHeader per Vcc, GND, RX e TX da connettere al microcontrollore.

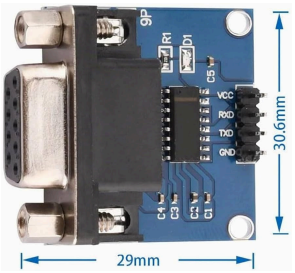


Figura 4: Traslatore di livello con MAX3232

2.3 Scheda PCB

Per facilitare l'assemblaggio di tutto il sistema è stato realizzato un circuito stampato appositamente per questo dispositivo, prodotto dalla JLCPCB a Shenzen (Cina). Dispone di uno screw terminal per l'attacco dell'alimentazione, due condensatori per stabilizzare la tensione, due file di PinSocket sulla quale innestare l'ESP32 e inoltre espone su dei PinHeader l'interruttore per l'alimentazione (SW1), il pulsante di reset (SW2), i pin della comunicazione seriale dell'ESP (J5) da connettere al modulo convertitore, e infine i bus SPI, I2C e ulteriori GPIO (General Purpose I/O) per eventuali espansioni o modifiche del sistema.

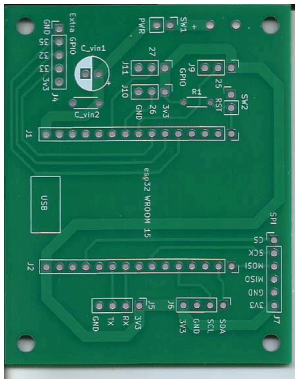


Figura 5: La PCB impiegata

2.4 Cavo USB-RS232

Sia per effettuare i test di comunicazione con il telescopio che per verificare il corretto funzionamento del dispositivo è stato utilizzato un cavo che ad un capo aveva un attacco USB, dall'altro un attacco DB-9 per il collegamento RS232 e un semplice circuito di conversione tra i due standard posto all'interno del connettore.

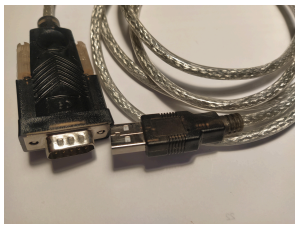


Figura 6: Cavo USB-RS232

2.5 HTTP

Hypertext Transfer Protocol (HTTP) è il protocollo a livello applicativo che, insieme a HTML e URL, costituisce l'infrastruttura di base del World Wide Web.
Questo strumento definisce le regole standard per lo scambio di informazioni in rete.
Dal punto di vista architetturale, HTTP opera secondo un modello client/server basato su un meccanismo di richiesta e risposta, ed è nativamente stateless, il che significa che non memorizza alcuna traccia degli scambi precedenti, trattando ogni interazione come completamente isolata e indipendente.
La richiesta del client specifica l'azione e il destinatario tramite un metodo HTTP e l'URL, accompagnati da intestazioni (headers) con i metadati e da un corpo (body) opzionale contenente i dati da inviare.
La risposta del server restituisce un codice di stato numerico (come 200 OK o 404 Not Found), nuove intestazioni che descrivono il tipo di file e il corpo del messaggio con la risorsa concreta richiesta (es. HTML o JSON).
Qui riportato un esempio di richiesta e risposta http

```
GET /wiki/Pagina_principale HTTP/1.1
Host: it.wikipedia.org
User-Agent: Mozilla/5.0 (compatible;
Konqueror/3.2; Linux) (KHTML, like
Gecko)
Accept: text/html, image/jpeg, image/
png, text/*, image/*, */*
Accept-Charset: iso-8859-1,
utf-8;q=0.5, *;q=0.5
Accept-Language: it
Connection: Keep-Alive
```

Codice 1: esempio di richiesta HTTP

```
HTTP/1.1 200 OK
Date: Fri, 22 Feb 2019 10:50:37 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 22208
Connection: keep-alive
Vary: Accept-
Encoding,Cookie,Authorization
[...]

<!DOCTYPE html>
[...]
</html>
```

Codice 2: esempio di risposta HTTP

2.6 API REST

Un'API di tipo Representational State Transfer (REST) è un'architettura software leggera e scalabile che permette a due sistemi di comunicare via HTTP in modo indipendente dalla rete fisica.
Si basa sul concetto di risorsa (un'astrazione di dati o funzionalità) identificata gerarchicamente tramite URI.
Le caratteristiche principali sono:

- **Risorse indirizzabili**
  - usare nomi descrittivi;
  - non usare verbi negli URL;
  - usare forme plurali per aggregati di risorse;
  - utilizzare i metodi HTTP per interagire con la risorsa.
- **Rappresentazioni**
  - La forma concreta con cui la risorsa viene scambiata (es. JSON o XML).

```
GET http://myRoom:6767/wardrobe/
tshirts

HTTP/1.1 200 OK
Content-Type: Application/json
[...]

{
  "tshirts": {
    "clean_count": 15,
    [...]
  },
  "links" : {
    "deposit":{
      "href": "/wardrobe/tshirts/
deposit",
      "method" : "POST"
    },
    [...]
  }

}
```

Codice 3: esempio di richiesta e risposta REST

2.7 TCP

Transmission Control Protocol (TCP) è un protocollo di rete a pacchetti a livello di trasporto. Appartiene alla suite di protocolli internet che si occupa di rendere una trasmissione dati affidabile.
TCP è un protocollo orientato alla connessione: ovvero prima di poter trasmettere dei dati è necessario stabilire, negoziando, una connessione punto-punto tra client e server. Dopodiché inizia uno scambio di flussi di bytes bidirezionali (full-duplex) con modalità di ritrasmissione in caso di errori.

2.8 PuTTY

PuTTY è un software per Windows che permette di connettere il computer come terminale a connessioni TCP tramite i vari standard SSH, Telnet, rLogin e a connessioni Seriali, accedendo alle varie porte di I/O. In questo caso è stato utilizzato per inviare le stringhe di comando al telescopio e ricevere la risposta, permettendo di modificare la BaudRate della connessione, l'encoding dei bit e e dei caratteri. (approfondito in seguito)

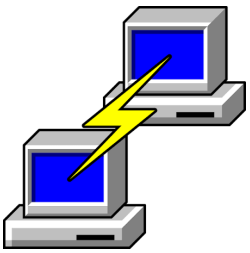


Figura 7: Icona di PuTTY

2.9 KiCAD

Il progetto del circuito stampato è stato realizzato con il software di Computer Assisted Design di schede elettroniche KiCAD, il quale permette all'utente di tracciare lo schema delle connessioni elettriche tra i vari componenti, potendo dividerlo in sezioni (come per esempio la sezione dell'alimentazione, la sezione del microcontrollore e altro) per facilitarne la lettura. Dopodiché il software contiene al suo interno una vasta libreria di impronte dei componenti da associare agli elementi presenti nello schema elettrico, in modo tale da poter disegnare il circuito stampato effettivo. Il software inoltre assiste l'utente nella creazione delle tracce di collegamento, si assicura che corrispondano ai collegamenti sullo schema elettrico e controlla che non ci siano collisioni o problemi. Una volta finito il progetto viene convertito in un file .gerber e inviato alla fabbrica per la sua realizzazione.

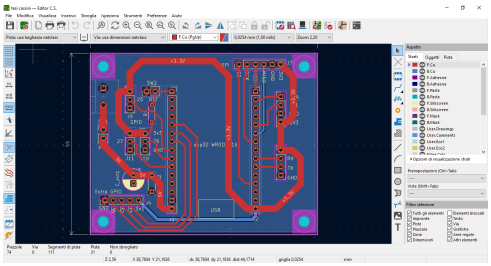


Figura 8: Programma KiCAD

2.10 Visual Studio Code

La scrittura del firmware è avvenuta con il software Microsoft Visual Studio Code, il programma di sviluppo software più diffuso, apprezzato per la sua semplicità nell'utilizzo e la possibilità di installare delle estensioni per ampliarne le capacità. In questo progetto è stata utilizzata l'estensione "PlatformIO", che permette di scrivere codice, scaricare librerie e effettuare l'upload del firmware sul microcontrollore direttamente dall'IDE. Per fare ciò sono stati utilizzati il linguaggio di programmazione C++ e l'ambiente di sviluppo Arduino adattato all'architettura EspressIf, azienda produttrice degli ESP32.

3 Architettura del sistema

L'osservatorio presenta quattro moduli principali che ne controllano le varie funzioni:

A. La cupola rotante
B. La feritoia per l'osservazione con il portellone apribile
C. Il telescopio
D. Il sistema di controllo centrale su Raspberry Pi

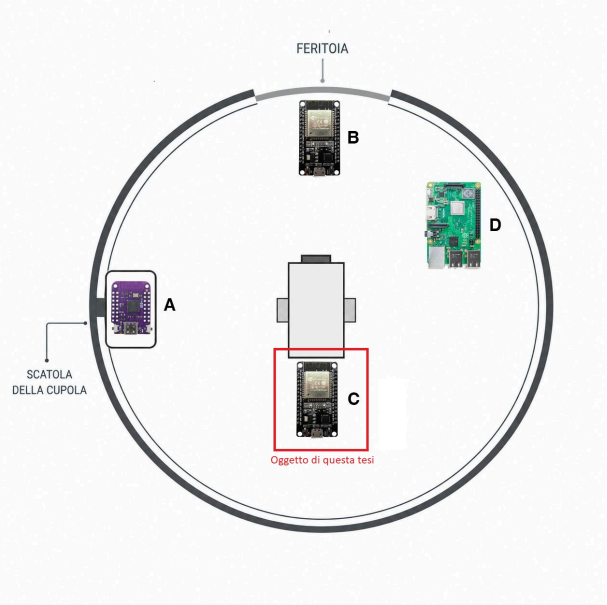


Figura 9: schema della planimetria dell'osservatorio

3.1 Topologia del sistema

Durante la progettazione iniziale sono state esplorate diverse opzioni per l'interconnessione e lo scambio di informazioni tra i moduli che compongono il sistema dell'osservatorio, ciascuna con pregi e difetti che sono stati attentamente considerati per definire quali sarebbero dovute essere le specifiche di un modulo. Qualsiasi tecnologia avessimo deciso di utilizzare avrebbe dovuto garantire determinate qualità:

- **Scalabilità**: il sistema deve permettere la facile installazione di eventuali nuovi moduli
- **Standard**: la comunicazione tra i vari moduli deve avvenire con modalità diffuse e normalmente utilizzate
- **Robustezza**: le connessioni devono essere affidabili e funzionanti
- **Semplicità**: tutti i moduli comunicano nella stessa maniera

Per garantire la robustezza e scalabilità del sistema è stata scelta una topologia a stella che prevede un sistema di controllo centralizzato, implementato nel Raspberry Pi, che ha il compito di ricevere gli input dall'utilizzatore, verificarli e distribuire i comandi da svolgere ai rispettivi moduli. Questo modello di rete permette anche di sviluppare i moduli in maniera indipendente, senza essere costretti ad intervenire sul resto del sistema quando bisogna effettuare aggiornamenti, modifiche o aggiunte. Inoltre, avere un singolo punto di accesso al sistema permette di modificare facilmente la modalità di invio degli input.

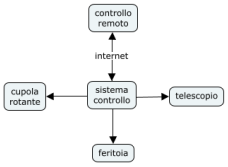


Figura 10: Topologia a stella del sistema

3.2 Modalità di interconnessione

Inizialmente era stata esplorata l'opzione di una connessione seriale cablata tra tutti gli elementi, questa soluzione ha però immediatamente presentato delle criticità. Nonostante infatti possa sembrare la via più semplice, questo approccio avrebbe richiesto la creazione di un protocollo ad hoc di invio e ricezione dei dati e soprattutto la posa dei cavi che avrebbe implicato importanti modifiche alla struttura; sarebbe stato infatti molto difficile inserire un cavo di connessione con il modulo di controllo della feritoia, posto sulla cupola rotante. Anche una soluzione ibrida seriale e wireless avrebbe compromesso il principio di semplicità a cui questo progetto è ispirato.
Si è scelto quindi di utilizzare unicamente connessioni wireless. Anche in questo ambito le opzioni possibili erano diverse. La prima implicava l'utilizzo di un'interfaccia puramente HTTP, inadatta allo scopo in quanto nativamente stateless e la presenza di parti meccaniche in movimento non era compatibile con questa architettura, che richiede risposte istantanee e indipendenti dalle richieste precedenti.
La seconda opzione era un ibrido "HTTP + WebSocket", ove le richieste ai moduli sarebbero state inviate tramite HTTP e le risposte differite nel tempo restituite tramite una connessione persistente WebSocket. Anche questa ipotesi è stata ritenuta inadatta a causa del limitato supporto al protocollo WebSocket fornito dall'azienda produttrice dei microcontrollori ESP32 EspressIf, che avrebbe posto significative complessità nello sviluppo dei firmware e il loro mantenimento.
Si è quindi scelto di utilizzare un'architettura "HTTP + TCP" sfruttando le caratteristiche di accessibilità e diffusione dell'HTTP e dei socket TCP nativi che, nonostante questi ultimi siano un protocollo a più basso livello rispetto al WebSocket, essi garantiscono comunque robustezza nella comunicazione, rendendo inoltre molto più facile la loro implementazione firmware.



Figura 11: Schema sintetico delle connessioni

Infine è stato scelto di sviluppare l'interfaccia HTTP dei singoli moduli secondo lo standard REST, che permette di creare interfacce uniformi e ben definite, per facilitare lo sviluppo dei sottosistemi. Per contrappasso le informazioni di ritorno è stato deciso di rappresentarle in formato JSON (JavaScript Object Notation), già sfruttato in numerosissimi ambiti di comunicazione e con un largo supporto software, grazie alla sua versatilità di utilizzo in diversi linguaggi di programmazione oltre al JavaScript per la quale era stato ideato, e alla sua facile leggibilità da parte di una persona.

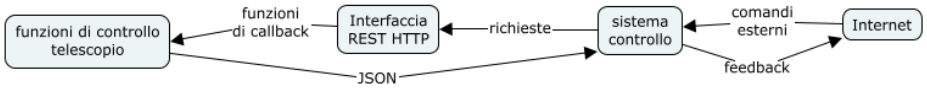


Figura 12: Catena di comando dall'utente al modulo

4 Il sistema telescopio

Come menzionato precedentemente all'interno dell'osservatorio astronomico è stato installato un telescopio Meade LX200GPS ACF da 12 pollici dotato di AutoStar II. Si tratta di un sistema a sè stante in grado di trovare automaticamente la propria collocazione geografica e calibrare la sua posizione su una delle stelle predefinite che riesce ad osservare dalla sua lente. Contiene altresì al suo interno una vasta libreria di oggetti astronomici da puntare, selezionabili grazie al telecomando AutoStar II, i quali una volta selezionati verranno seguiti automaticamente in base alla rotazione terrestre.



Figura 13: Topologia a stella del sistema

Il telescopio è anche provvisto di una messa a fuoco automatica che è al momento inutilizzata. Nell'impiego ideato in questo progetto è stato deciso di semplificare le mansioni che deve eseguire il firmware del telescopio nella prospettiva futura di utilizzare software che darà le coordinate desiderate dall'utente remoto, come per esempio Stellarium o programmi analoghi. Questi programmi sono stati creati con lo scopo di effettuare a loro volta il tracciamento degli oggetti astronomici data una posizione geografica e controllare un dispositivo di osservazione, rendendo quindi superflue tali funzionalità del telescopio.
Il telescopio deve quindi limitarsi a inviare informazioni riguardo la propria posizione e modificarla quando richiesto.

4.1 La porta seriale RS232

Il telescopio dispone di una porta seriale RS232 per comunicare con un dispositivo di controllo, tramite un protocollo sviluppato dall'azienda produttrice stessa, approfondito nella prossima sezione. Questo standard di livello fisico dello stack ISO/OSI è stato creato nella seconda metà del ventesimo secolo per lo scambio di dati fra dispositivi digitali "full duplex", ovvero la trasmissione e la ricezione delle informazioni può avvenire simultaneamente (caratteristica di qualsiasi connessione seriale), anche se in questa applicazione non accade mai.



Figura 14: Pannello I/O del telescopio e porta seriale

Diversamente da una connessione seriale TTL (standard nei moderni dispositivi elettronici, ESP32 incluso) che associa al livello logico "0" una tensione di 0v, e al livello logico "1" una tensione pari alla VCC di riferimento (5v per Arduino, 3.3v per ESP32), l'RS232 associa al livello logico "0" una tensione positiva e al livello logico "1" una tensione uguale ma negativa.

Quest'ultima varia in base all'applicazione in quanto lo standard impone solo un tetto massimo di |24|v. Risulta evidente che non sia possibile collegare direttamente l'RS232 al microcontrollore ESP32, di conseguenza bisogna inserire un traslatore di livello. In commercio ne sono disponibili diversi modelli, ne è stato scelto un tipo che monta il chip MAX3232 che funziona alla tensione di 3.3v dell'ESP32, ha un costo molto contenuto, e viene fornito già assemblato con il connettore DB-9, il più comune per lo standard RS232.

Il livello di linea è gestito dallo standard UART (Universal Asincronous Reciever-Trasmitter), periferica all'interno dell'ESP32 (e rispettivamente nel microcontrollore del telescopio) che gestisce, come si evince dal nome, l'invio e la ricezione dei dati.

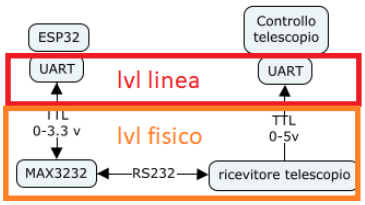


Figura 15: Schema livello fisico e di linea

Andando ad analizzare la codifica dei bit si osserva che a riposo la linea presenta un livello logico "1", successivamente, quando inizia lo scambio dei dati, viene inviato in maniera asincrona un bit di start "0", ove la commutazione del segnale elettrico determina il segnale di sincronizzazione, poi viene inviato il messaggio dal LSB al MSB, per terminare infine in dei bit di stop a livello logico "1" e la linea torna a riposo.

Di fatto, la periferica UART è un convertitore di bit paralleli in arrivo dal BUS interno al microcontrollore che li invia poi in maniera seriale sulla linea TTL. Per fare ciò i dati vengono formattati in diversi formati di "frame", i pacchetti di bit, che possono presentare 7 o 8 bit di dato, un bit di parità e 1 o 2 bit di stop. Per decifrare infine i bit in arrivo dopo la prima commutazione del bit di start, deve essere imposta una BaudRate, ovvero quanto deve durare un bit prima di commutare nel prossimo bit.

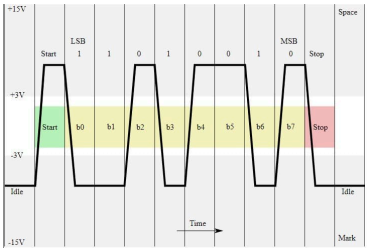


Figura 16: Grafico segnale elettrico RS232

Nel caso del telescopio il frame è formattato in 8N1, ovvero 8 bit di dati, nessun bit di parità e 1 bit di stop, alla velocità di 9600 baud, un bit ogni 104 uS. La formattazione del frame e la velocità vanno impostati a priori su entrambi i dispositivi coinvolti nel collegamento, in quanto lo standard non prevede alcun tipo di handshake o scambio di informazioni preliminare.

4.2 Il protocollo Meade LX200

Una volta stabilita la connessione tra il microcontrollore e il telescopio, è necessario effettuare lo scambio di informazioni attraverso stringhe di testo definite nel documento "Meade Telescope Serial Command Protocol" fornito dall'azienda produttrice. I comandi sono suddivisi in diverse categorie in base al loro scopo e agli accessori installati, come ad esempio la messa a fuoco automatica o il GPS. Generalmente i telescopi controllati elettronicamente si allineano con gli oggetti celesti da osservare con due coordinate: L'Ascensione Retta (da adesso chiamata RA) e l'Angolo di Declinazione (da adesso chiamato DA), che devono essere calcolate in base alla posizione geografica del telescopio sulla Terra.

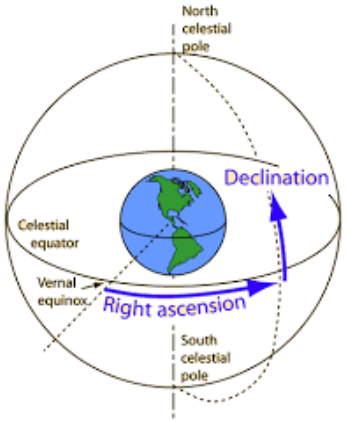


Figura 17: Schema coordinate AltAzimutali

I comandi utilizzati in questo progetto sono:

Comandi configurazione:	Note	Sintassi della stringa	Stringa di risposta
ACK	Bit di ack	0x06 esadecimale	A, L, P in base alla modalità di allineamento
Calibrate Mount	Comando di calibrazione	:CM#	Stringa statica della galassia di andromeda M31 EX GAL MAG 3.5 SZ178.0'#
Toggle Precision	Cambia da precisione alta a bassa e viceversa	:U#	nessuna
Force AltAz aligment	Imposta la modalità di allineamento a AltAzimutale	:AA#	nessuna

Figura 18: Tabella comandi di configurazione

Comandi di movimento:	Note	Sintassi della stringa	Stringa di risposta
Move/Slew	Avvia il movimento del telescopio verso le coordinate obbiettivo impostate	:MS#	0 – OK 1 – oggetto sotto il limite minimo di declinazione 2 – oggetto sopra il limite massimo di declinazione
Quit	Freno di emergenza	:Q#	nessuna
Initialize	Inizializza il telescopio alla posizione di default	:I#	nessuna

Figura 19: Tabella comandi di movimento

Comandi informazioni	Note	Sintassi della stringa	Stringa di risposta
Get Right Ascension Angle	Ottieni la coordinata RA corrente	:GR#	:DD:MM:SS#
Get Declination Angle	Ottieni la coordinata DA corrente***	:GD#	:DD°MM:SS#
Get Lower Limit	Ottieni l'angolo minimo alla quale il telescopio può andare	:Go#	:DD*#

Figura 20: Tabella comandi di richiesta informazioni

Comandi Coordinate obbiettivo	Note	Sintassi della stringa	Stringa di risposta
Set Right Ascension angle	Imposta la coordinata RA	:SrDD:MM:SS#	0 – non valida 1 - valida
Set Declination Angle	Imposta la coordinata DA, (s) è il segno positivo o negativo dell'angolo	:Sd(s)DD:MM:SS#	0 – non valida 1 – valida

Figura 21: Tabella comandi di impostazione coordinate obbiettivo

4.3 Nota sulla codifica dei caratteri

Durante il lavoro di decodifica delle informazioni ricevute è sorto un problema legato alla decodifica dei dati "crudi" in arrivo dalla periferica UART in caratteri codificati legato a problemi di retrocompatibilità con le vecchie librerie di codifiche. Dalle prove in loco della comunicazione con il telescopio è stato osservato che il simbolo dei gradi "°", presente nelle stringhe contenenti le coordinate dell'angolo di Declinazione, appariva sul terminale con il simbolo "", simbolo utilizzato dall'interprete quando un carattere presenta degli errori o non viene riconosciuto. Ciò è causato dalla scorretta interpretazione del carattere da parte del programma di terminale, e per spiegare questo fenomeno bisogna andare ad analizzare le due librerie di caratteri diverse utilizzate dai due sistemi, HD44780 (libreria di caratteri per i display lcd prodotti dall'Hitachi) utilizzata dal telescopio e l'UTF-8 del programma terminale.
In entrambe le codificazioni i primi 128 caratteri sono codificati in un byte come nell'ASCII standard a 7 bit, infatti le stringhe di comando e la maggior parte dei caratteri sono letti e inviati senza problemi. La libreria dell'HD44780 tuttavia aggiunge altri 128 caratteri codificandoli in 8 bit, mentre l'UTF-8 codifica i restanti 140.000+ caratteri della sua libreria con una coppia di due byte. Il byte 0xDF viene quindi visto dall'interprete UTF-8 come la "prima parte" di un altro carattere, che tuttavia non esiste.

Carattere da leggere	Esadecimale inviato dal telescopio	Codifica per HD44780	Codifica UTF-8	Codifica Extended ANSI
°	0xDF	°	???	ß

Figura 22: Diversa interpretazione del byte

In questa tabella (Fig. 22) viene mostrato ciò che dovrebbe essere letto dall'utente, il suo valore esadecimale e le varie interpretazioni delle diverse librerie di caratteri. È presente anche come verrebbe interpretato dalla codifica Extended ANSI per far notare che anche una codifica della stessa epoca possa dare un risultato diverso da quello atteso, interpretando il byte come il carattere "ß" dell'alfabeto tedesco. Quello che potrebbe sembrare un insignificante equivoco avrebbe determinato la totale inutilizzabilità del dispositivo, evidenziando l'enorme importanza di una documentazione corretta durante progettazione dei sistemi.

Table 4 Correspondence between Character Codes and Character Patterns (ROM Code: A00)



Note: The user can specify any pattern for character-generator RAM.

The ANSI Character Set

Code	Char	Name	Code	Char	Name
128	€	unused	192	À	A grave
129		unused	193	Á	A acute
130	‚	baseline single quote	194	Â	A circumflex
131	ƒ	florin	195	Ã	A tilde
132	„	baseline double quote	196	Ä	A diaeresis
133	…	ellipsis	197	Å	A ring
134	†	dagger (single)	198	Æ	AE ligature
135	‡	dagger (double)	199	Ç	C cedilla
136	ˆ	circumflex	200	È	E grave
137	‰	per mil	201	É	E acute
138	Š	S caron	202	Ê	E circumflex
139	‹	left single guillemet	203	Ë	E diaeresis
140	Œ	OE ligature	204	Ì	I grave
141		unused	205	Í	I acute
142	Ž	unused	206	Î	I circumflex
141		unused	205	Í	I acute
142	Ž	unused	206	Î	I circumflex
143		unused	207	Ï	I diaeresis
144		unused	208	Ð	Icelandic Eth
145	‘	open single quote	209	Ñ	N tilde
146	’	close single quote	210	Ò	O grave
147	“	open double quote	211	Ó	O acute
148	”	close double quote	212	Ô	O circumflex
149	•	bullet (large)	213	Õ	O tilde
150	–	en dash	214	Ö	O diaeresis
151	—	em dash	215	×	multiply symbol
152	˜	tilde	216	Ø	O with oblique stroke
153	™	unregistered trademark	217	Ù	U grave
154	š	s caron	218	Ú	U acute
155	›	right single guillemet	219	Û	U circumflex
156	œ	oe ligature	220	Ü	U diaeresis
157		unused	221	Ý	Y acute
158	ž	unused	222	Þ	Icelandic Thorn
159	Ÿ	Y diaeresis	223	ß	German sharp s
160		non-breaking space	224	à	a grave
161	¡	Spanish inverted !	225	á	a acute
162	¢	cents	226	â	a circumflex
163	£	pounds	227	ã	a tilde
164	¤	intl. monetary symbol	228	ä	a diaeresis
165	¥	yen	229	å	a ring
166	¦	broken bar	230	æ	ae ligature
167	§	section symbol	231	ç	c cedilla
168	¨	diaeresis	232	è	e grave
169	©	copyright	233	é	e acute
170	ª	feminine ordinal	234	ê	e circumflex
171	«	left double guillemet	235	ë	e diaeresis
172	¬	not	236	ì	i grave
173		soft hyphen	237	í	i acute
174	®	registered trademark	238	î	i circumflex
175	¯	macron	239	ï	i diaeresis
176	°	ring (also degrees)	240	ð	Icelandic eth
177	±	plus/minus	241	ñ	n tilde
178	²	superscript 2	242	ò	o grave
179	³	superscript 3	243	ó	o acute
180	´	acute	244	ô	o circumflex
181	µ	micro symbol (or mu)	245	õ	o tilde
182	¶	pilcrow (paragraph symbol)	246	ö	o diaeresis
183	·	bullet (small)	247	÷	divide symbol
184	¸	cedilla	248	ø	o with oblique stroke
185	¹	superscript 1	249	ù	u grave
186	º	masculine ordinal	250	ú	u acute
187	»	right double guillemet	251	û	u circumflex
188	¼	one-fourth	252	ü	u diaeresis
189	½	one-half	253	ý	y acute
190	¾	three-fourths	254	þ	Icelandic thorn
191	¿	Spanish inverted ?	255	ÿ	y diaeresis

5 Il firmware del microcontrollore

Il firmware del microcontrollore è organizzato su un unico file di configurazione denominato "serial_config.h". Esso contiene tutte le variabili globali, i #define, le inclusioni delle librerie esterne e i prototipi delle funzioni di gestione della rete, del telescopio e delle funzioni di callback HTTP, separati nelle loro rispettive categorie di competenza.

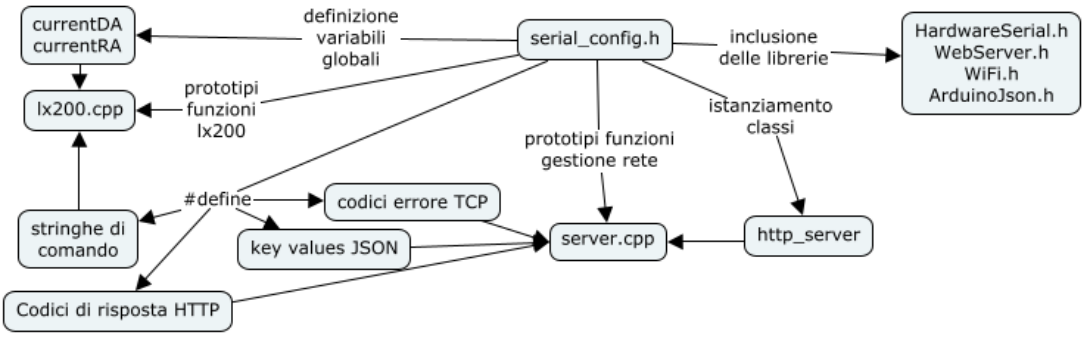


Figura 23: Struttura del firmware

5.1 Controllo del telescopio

Struct AltAz

Per la memorizzazione delle coordinate AltAzimutali è stata creata la struct altaz_t che contiene tre variabili uint8_t rispettivamente per le Ore, Minuti e Secondi che compongono la coordinata. Inoltre, è presente anche una variabile booleana per il segno, necessaria per la gestione della coordinata DA, la quale può avere anche segno negativo. Sono quindi istanziate come variabili globali le struct AltAz currentRA per la memorizzazione della coordinata dell'ascensione Retta e la struct AltAz currentDA per la coordinata dell'angolo di Declinazione. Queste variabili sono globali per permetterne il facile accesso a tutte le funzioni di gestione della posizione, tuttavia vengono modificate solo dalle funzioni di richiesta.

```
typedef struct altaz_t {
    bool positive;
    uint8_t hours;
    uint8_t minutes;
    uint8_t seconds;
    altaz_t(){  //costructor
        positive = true;
        hours = minutes = seconds = 0;
    }
}AltAz;
```

Figura 24: La dichiarazione della struct

Funzioni LX200

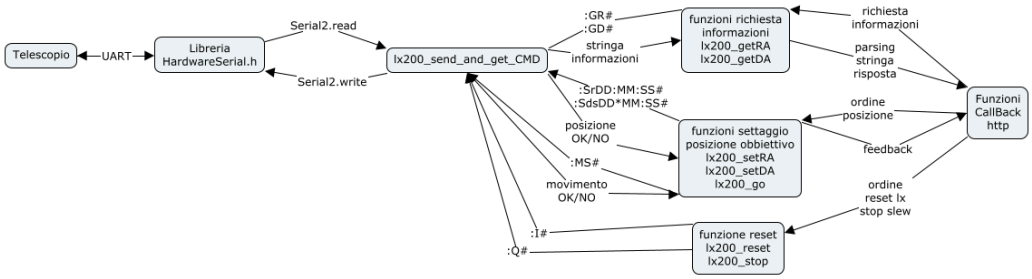


Figura 25: Schema delle funzioni di controllo del telescopio

La gestione delle comunicazioni seriali con il telescopio avviene grazie alla libreria HardwareSerial appartenente al framework di sviluppo Arduino. Questa libreria permette la lettura e l'inivio dei dati dalla periferica UART della seriale del microcontrollore tramite le funzioni Serial2.read per la lettura e Serial2.write o Serial2.print per l'invio.
L'uso di queste funzioni di libreria è esclusivo alla funzione lx200_send_and_recieve_CMD, che svolge il ruolo di inviare la stringa di comando sulla periferica UART e riceve la stringa di risposta se richiesta e controlla che la sintassi sia corretta in caso di errori durante la ricezione. La risposta può variare in base al comando inviato; i comandi di richiesta della posizione, per esempio, devono contenere il carattere di fine stringa "#" definita dal protocollo di comunicazione Meade LX200, mentre altri, come l'invio delle coordinate obbiettivo, non lo contengono. L'attivazione del controllo del carattere di fine stringa è determinato dalla variabile booleana "endswithpound". È presente anche una versione in overloading che permette l'invio di singoli byte per la trasmissione del bit di ACK.
Tutte le altre funzioni di controllo del telescopio utilizzano la funzione lx200_send_and_recieve_CMD per inviare la specifica stringa di comando dedicata al loro scopo, ed effettuano il controllo della sintassi e il parsing delle informazioni ricevute nella struct AltAz. Tutte le funzioni lx200 sono state implementate e documentate nel file lx200.cpp

Le stringhe di comando

Le stringhe di comando sono memorizzate in dei #define sempre all'interno del file serial_config.h, rendendone facile l'utilizzo senza il rischio di errori di battitura e rendendoli facilmente leggibili. I comandi utilizzati e il loro scopo sono elencati nella sezione 4.2. Le stringhe per l'impostazione delle coordinate obbiettivo invece sono "assemblate" nelle funzioni lx200_set_XX, dato che le stringhe da inviare per svolgere questa funzione variano in base ai valori delle coordinate obbiettivo, e per gestire il carattere "°" problematico.

```
char cmd[24] = {0};
snprintf(cmd, sizeof(cmd), ":Sd%c%02d%c%02d:%02d#",
         sign,
         (int)coords.hours,
         (char)0xDF, // '°' char
         (int)coords.minutes,
         (int)coords.seconds);
```

Figura 26: «montaggio» della stringa di comando

5.2 Gestione della rete

La gestione della connessione WiFi, dell'interfaccia http e l'invio dei pacchetti TCP è gestita da funzioni dedicate implementate nel file server.cpp, che permettono il controllo remoto del telescopio. La dinamica della comunicazione, come già evidenziato in precedenza, è basata su un'architettura ibrida ove la ricezione dei comandi avviene tramite una REST API HTTP, mentre le informazioni in transito dal modulo al sistema di controllo centrale sono gestite tramite TCP in maniera disaccoppiata. L'esecuzione dei comandi con successivo invio delle informazioni avvengono quindi in un momento diverso rispetto all'arrivo dei comandi, ottimizzando così l'interazione tra i due dispositivi.

Interfaccia REST API

Il microcontrollore utilizza la libreria WebServer.h per esporre un server HTTP nativo: le richieste provenienti dal controllo centrale vengono mappate su endpoint mediante le funzioni di callback, le quali verificano in maniera preliminare la validità dei parametri negli ordini ricevuti, rispondono in maniera sincrona con i codici di stato standard del protocollo, poi effettuano il parsing delle informazioni e chiamano le funzioni di controllo del telescopio per svolgere le mansioni richieste. Anche i codici di errore sono definiti nel file serial_config.h, corredati con una spiegazione del loro caso d'uso. In questa tabella è sintetizzato l'albero dell'interfaccia rest con le relative funzioni di GET e PUT.

/network	/tcp_socket	GET: legge IP e porta
		PUT: imposta IP e porta
/settings	/resetESP	PUT: reset software dell'esp32
	/resetLX	PUT: invia comando :I#
	/heartbeat	GET: controllo connessione
position	/home	PUT: porta il telescopio alla posizione di default
	RA	GET: ottieni coordinata RA attuale
		PUT: imposta coordinata RA obbiettivo e avvia il movimento
	DA	GET: ottieni coordinata DA attuale
		PUT: imposta coordinata DA obbiettivo e avvia il movimento
	Query ?RA="ra"&DA="da"	GET: ottieni sia coordinata RA che coordiata DA
		PUT: imposta simultaneamente coordinata obbiettivo RA e DA e avvia il movimento

Figura 27: Schema della struttura REST API

Trasmissione dei dati asincrona via TCP e JSON

Per accomodare i tempi di latenza del telescopio nello svolgere le operazioni di gran lunga superiori ai tempi di risposta sincroni necessari all'interfaccia HTTP è stato utilizzato un socket TCP client nativo. I payload con le informazioni ed eventuali messaggi di errore sono serializzati in oggetti JSON, i quali poi vengono trasmessi come stringhe grezze per poi essere ricostruiti alla ricezione dal sistema di controllo centrale. Queste stringhe sono strutturate con il carattere terminatore hardware della stringa "$", come indicato nei #define, per assicurare la corretta ricezione e parsing delle informazioni.

```
{
  "key-stringa" = "testo",
  "key-numero"  = 67,
  "key-booleana" = true,
  "key-array"   = ["testo", 67]
}
```

Codice 4: Sintassi di un file JSON

I file JSON sono strutturati con l'accoppiamento di "key" e "values" e il rispettivo parametro, questi "key" rappresentano ogni tipo di informazione che il modulo del telescopio potrebbe dover inviare, e i "values" i parametri ad essi associati, ad esempio lo stato J_SERIAL_STATUS (fondamentale nell'architettura del sistema di controllo centrale, il quale segnala immediatamente un errore in caso questa key sia mancante o il parametro sia diverso da "ok"), le coordinate o gli errori con relativi motivi e codici di errore, separati in errori del sistema ed errori relativi all'impostazione delle coordinate obbiettivo. Per garantire una maggiore flessibilità alla connettività alla rete, è stata implementata la funzione di callback http_config_socket_PUT che consente al sistema di controllo centrale di aggiornare durante l'uso l'indirizzo IP e e la porta utilizzata dal socket di destinazione.

6 Contributo personale e considerazioni conclusive

Il lavoro svolto in questi mesi per la realizzazione di questo progetto è stato un crocevia di argomenti trattati nei vari corsi della laurea triennale; si sono toccati principalmente argomenti di sviluppo dei sistemi embedded per quanto riguarda la scrittura del firmware e la scelta dei componenti, ma sono stati altrettanto fondamentali quelli di telematica per lo studio delle comunicazioni e della gestione dei dati in transito tra i vari elementi che compongono questo sistema. Infine sono stati importanti anche insegnamenti meno strettamente correlati allo studio dei sistemi elettronici, per esempio le nozioni di base su spazi a più coordinate mi hanno aiutato a comprendere il sistema di funzionamento del puntamento dei telescopi, del quale non mi ero mai occupato prima d'ora.

Il progetto ha presentato delle importanti difficoltà nella decifrazione di vecchia documentazione e standard ormai obsoleti, ciò ha comportato la necessita di informarsi su tali modelli anche da fonti meno facilmente reperibili come forum di appassionati di astronomia, i cui utenti si sono spesso trovati nelle mie stesse condizioni. Questa situazione ha stimolato in particolare un mio specifico interesse personale nei confronti di tecnologie che possono essere considerate ormai storiche e riportarle almeno parzialmente alle loro condizioni operative originarie. Grazie a questa aggiunta al sistema complessivo di controllo del telescopio viene inserito un ulteriore tassello verso l'obbiettivo finale di rendere accessibile in maniera totalmente remota tutte le funzionalità dell'osservatorio. Questa potenzialità è di particolare importanza proprio per le funzioni didattiche e scientifiche che può fornire a studenti, professori, e ricercatori che potranno utilizzarlo. Il lavoro naturalmente dovrà proseguire nei prossimi anni. Gli obbiettivi da raggiungere quindi per chi sarà impegnato nella prosecuzione dei lavori sono quattro, di diverso tipo e natura progettuale:

- l'interfaccia di controllo remoto e l'adeguamento a programmi pre-esistenti per la gestione dell'osservazione degli oggetti astronomici;
- l'installazione della telecamera astronomica già acquistata dalla scuola, e l'invio del suo output video sulla rete;
- l'automatizzazione dell'apertura e chiusura del portellone della feritoia, già in fase di sviluppo;
- la configurazione e impostazione di un access point con la sua relativa sottorete locale per l'interconnessione di tutti questi elementi e permetterne un accesso a internet sicuro dagli attacchi informatici;

In conclusione, penso di poter dire che questa esperienza è stata per me altamente formativa, e credo che mi abbia offerto la possibilità di costruire solide basi per il mio futuro di studi e di lavoro.

Riferimenti Bibliografici

[1] espressif, «esp32 devkit V1 datasheet». [Online]. Disponibile su: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp-dev-kits-en-master-esp32.pdf

[2] texas instruments, «MAX 3232 datasheet». [Online]. Disponibile su: https://www.ti.com/lit/ds/symlink/max3232.pdf?ts=1791721456389&ref_url=https%253A%252F%252Fwww.google.com%252F

[3] Wikipedia, «REST». [Online]. Disponibile su: https://it.wikipedia.org/wiki/Representational_state_transfer

[4] Wikipedia, «Hypertext Transfer Protocol». [Online]. Disponibile su: https://it.wikipedia.org/wiki/Hypertext_Transfer_Protocol

[5] Meade Instruments, «Meade Telescope Serial Command Protocol». [Online]. Disponibile su: https://interactiveastronomy.com/lx-200gps_telescope_protocol_2010-10.pdf

[6] Wikipedia, «EIA RS-232». [Online]. Disponibile su: https://it.wikipedia.org/wiki/EIA_RS-232

[7] Alessandro Maryni, «Sistema embedded per controllo e movimentazione di osservatorio astonomico». [Online]. Disponibile su: https://github.com/Osservatorio-LISA/system-controller